UWE Bristol | University of the West of England

Presentation by

Majid Mumtaz

Continuous Integration Continuous Deployment Configuration Management

UFCF8S-30-2

What have you covered so far?

- Introduction and Software development methodologies
- System Analysis and Design process
- Object Oriented Modelling & Analysis
- SOLID principles
- Design Patterns

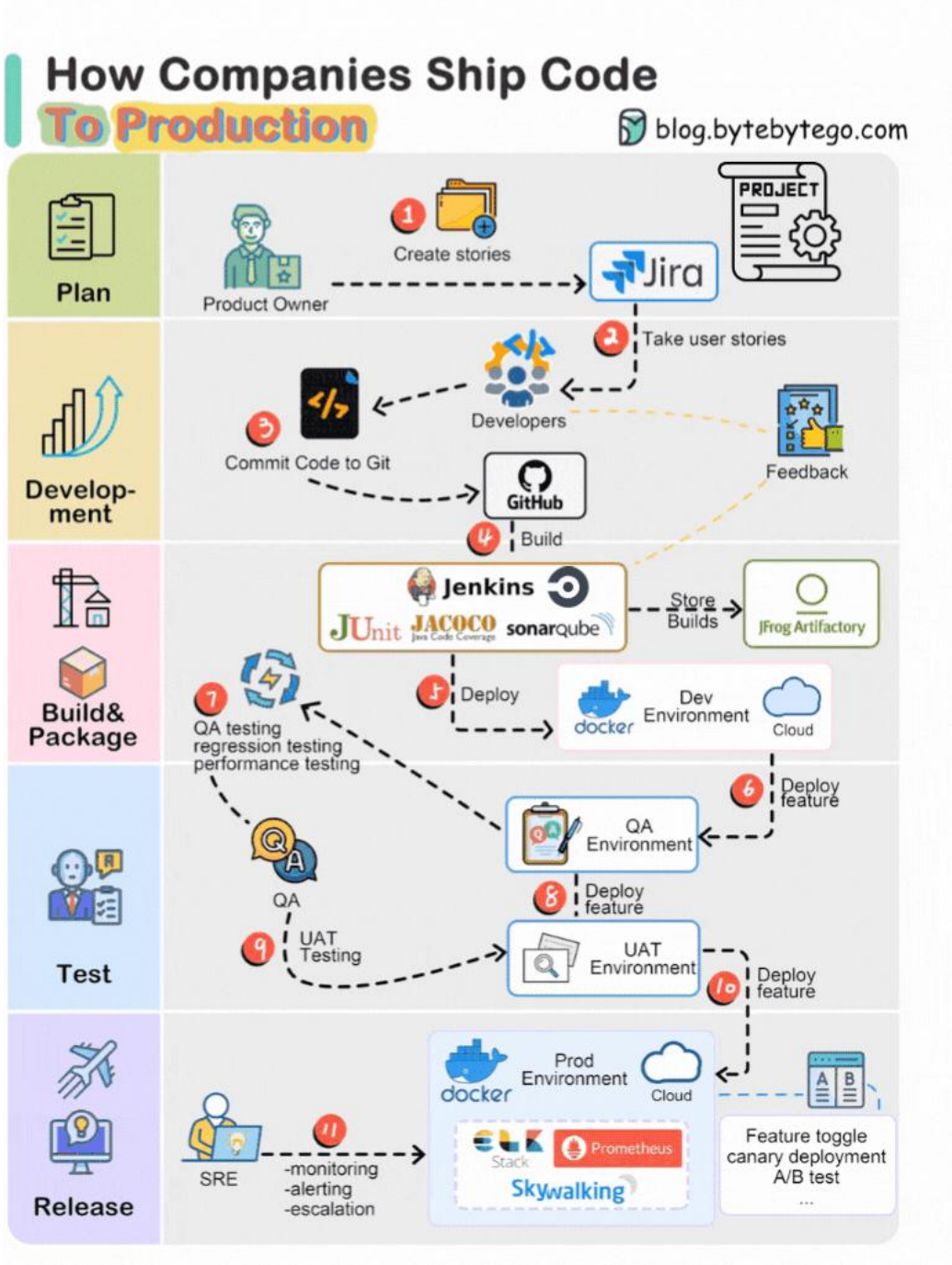
What we'll cover today?

- Automating development tasks
- Continuous Integration / Continuous Deployment
- Configuration Management

Why Automation?

- You looked at several software development methodologies
  - What was DevOps?

- Can SW development team(s) achieve any benefit if they automate the workflow of feature(s) development, integration, testing and deployment?



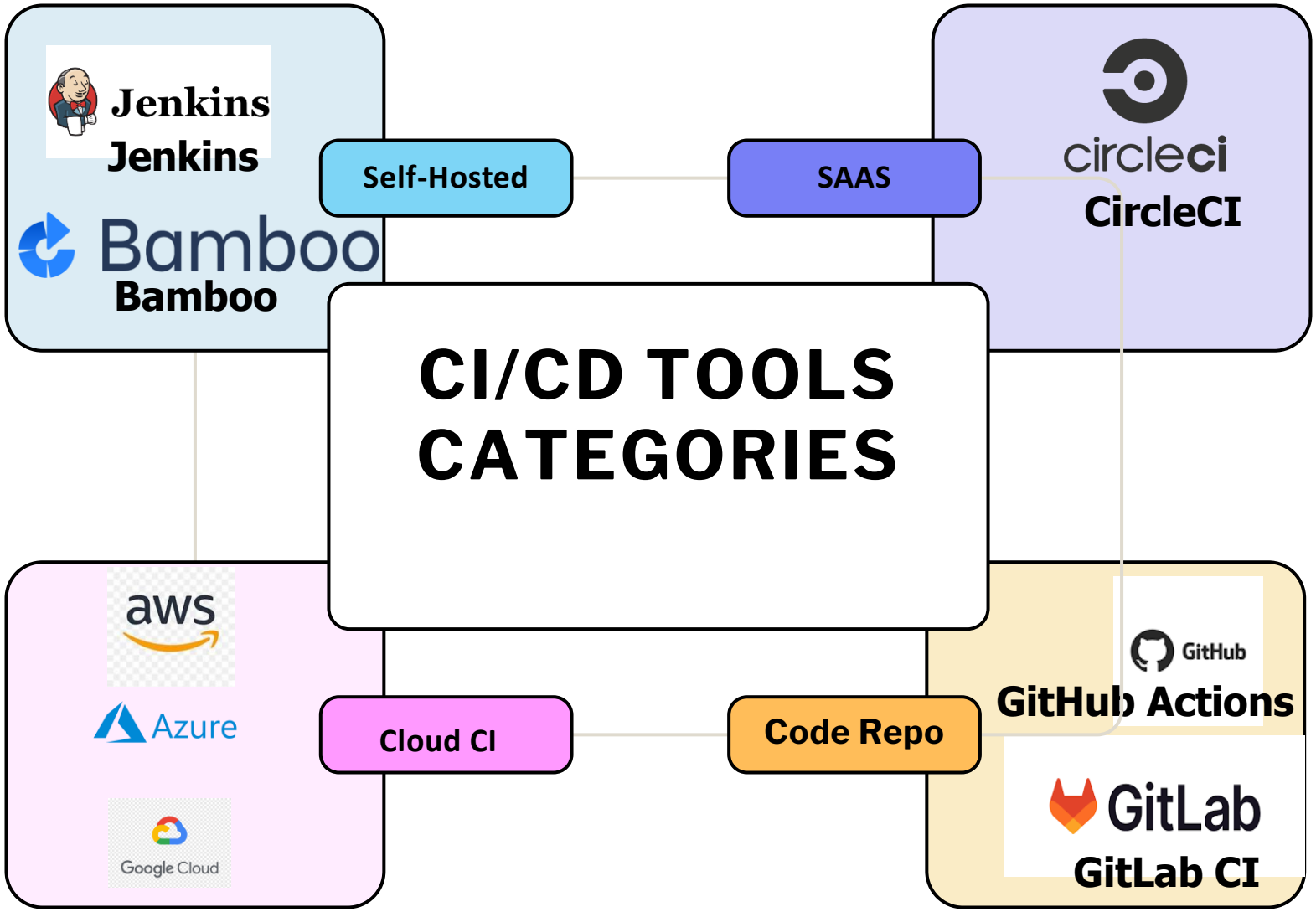
- Source: https://blog.bytebytego.com/p/ep81-how-companies-ship-code-to-production

Techniques and Tools

- **Continuous Integration (CI):**
  - Changing/updating code, developing new feature
  - When you push the code change, is the state of the code still OK?
  - Does it build/compile?
  - Does it pass the testing?

- **Continuous Deployment (CD):**
  - After code change and CI checks, deploy the code in production environment (e.g., deploy a server to AWS/Heroku, or run one-time job e.g., generate a report, clean database etc.

Techniques and Tools



Example: CircleCI

- CircleCI: https://circleci.com/
- CircleCI is a Continuous Integration and Continuous Delivery platform that supports rapid software development and publishing
- Can be used with agile method like **Scrum & DevOps**
- Can start for free (with restrictions) and build fast CI pipeline
- A **CI/CD pipeline** is a series of steps that must be performed before delivering a new version of software
- Is suitable for Git repositories like GitHub, GitLab or Bitbucket

CircleCI

circleci OPERATIONAL

ASD-UWE

- Organization Home
- Pipelines
- Projects
- Deploys NEW
- Insights
- Self-Hosted Runners
- Organization Settings
- Plan UPGRADE

All Pipelines

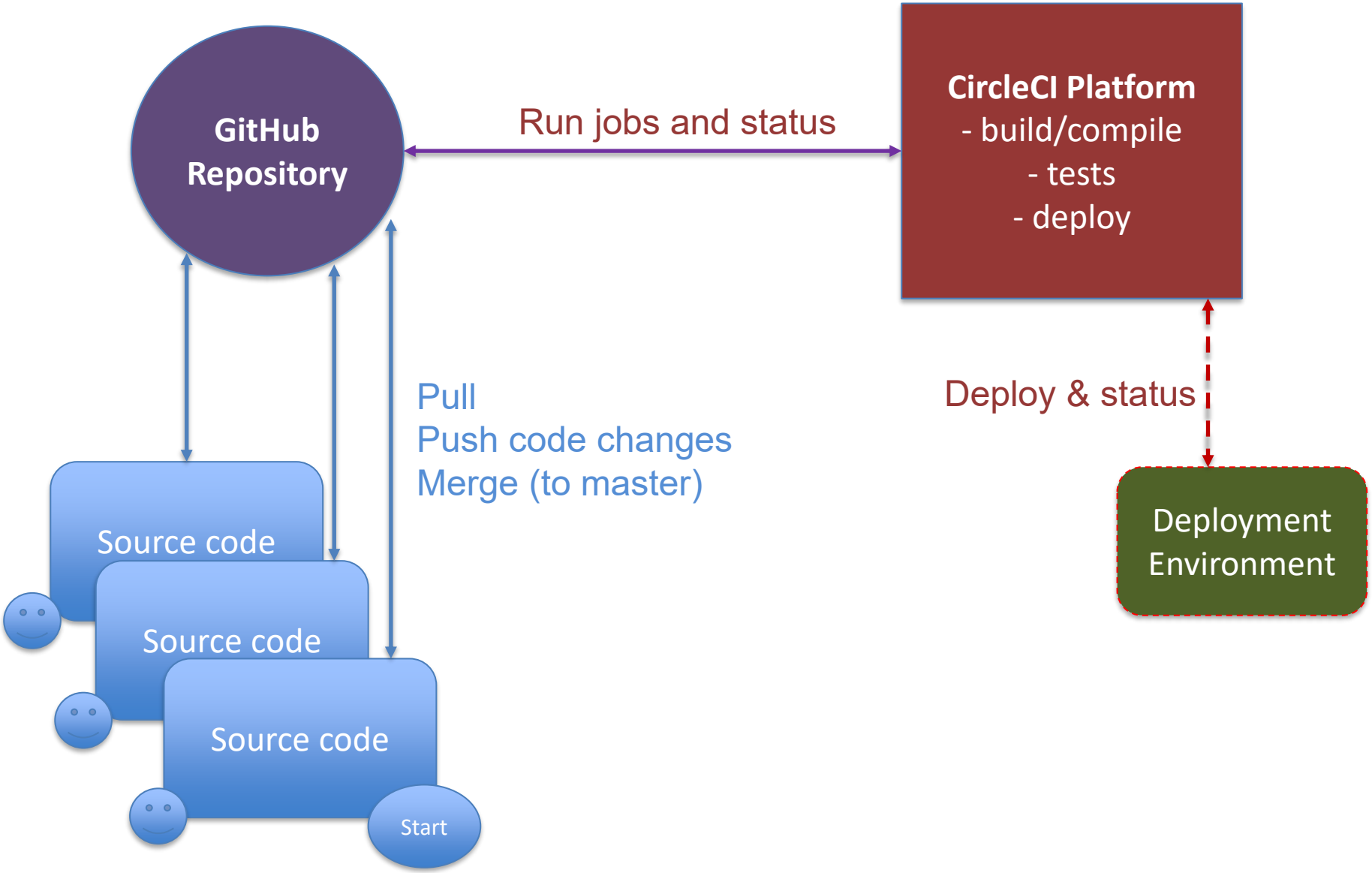
Everyone's Pipelines | All Projects | Select a Branch | All days | Auto-expand

Pin	Project	Status	Workflow	Trigger Event	Start	Duration	Actions
	circleci-demo 6	Success	build_and_test_and_deploy	MA main 4866e07 Update main.py	9m ago	1m 28s	
	circleci-demo-project 18	Success	build_and_test_and_deploy	MA main 4866e07 Update main.py	9m ago	1m 4s	
	circleci-demo 5	Success	build_and_test_and_deploy	MA main d02fb24 Update config.yml	11m ago	1m 4s	
	circleci-demo-project 17	Success	build_and_test_and_deploy	MA main d02fb24 Update config.yml	11m ago	1m 47s	
	circleci-demo 4	Success	build_and_test_and_deploy	MA main 939c8f9 Update main-test.py	13m ago	47s	
	circleci-demo-project 16	Success	build_and_test_and_deploy	MA main 939c8f9 Update main-test.py	13m ago	26s	
	circleci-demo 3	Failed	build_and_test_and_deploy	MA main bcd338b Update main-test.py	14m ago	15s	
	circleci-demo-project 15	Failed	build_and_test_and_deploy	MA main bcd338b Update main-test.py	13m ago	47s	

A typical CI/CD pipeline

- **Source**
  - Source code repository
  - Source code changes and commit to share code via repository
  - Activate a notification in the CI/CD pipeline tool
- **Build**
  - Updated code is shared to build (compile) a runnable instance of the product
  - Usually **docker containers** are used to deploy software
  - If software fails then there must be something fundamentally wrong with the configuration
- **Test**
  - Test (automated testing) and validate the code
  - Failure at this stage exposes bugs and errors which must be corrected while fresh in the minds of developers
- **Deliver/Deploy**
  - After successful built and testing i.e., there are no errors then deploy
  - Configure deployment job/pipeline

A typical CI/CD pipeline



Source code (main.py)

main.py M × | main-test.py | config.yml

main.py > ...

```python
def Add(a, b):
        return a + b

def Subtract(a, b):
        return a - b

def CallHello():
        print("hello world from cicircle tutorial")

if __name__ == '__main__':
        CallHello()
```

Example adapted from: https://www.srcmake.com/home/circleci

Source code (main-test.py)

main.py M | main-test.py × | config.yml

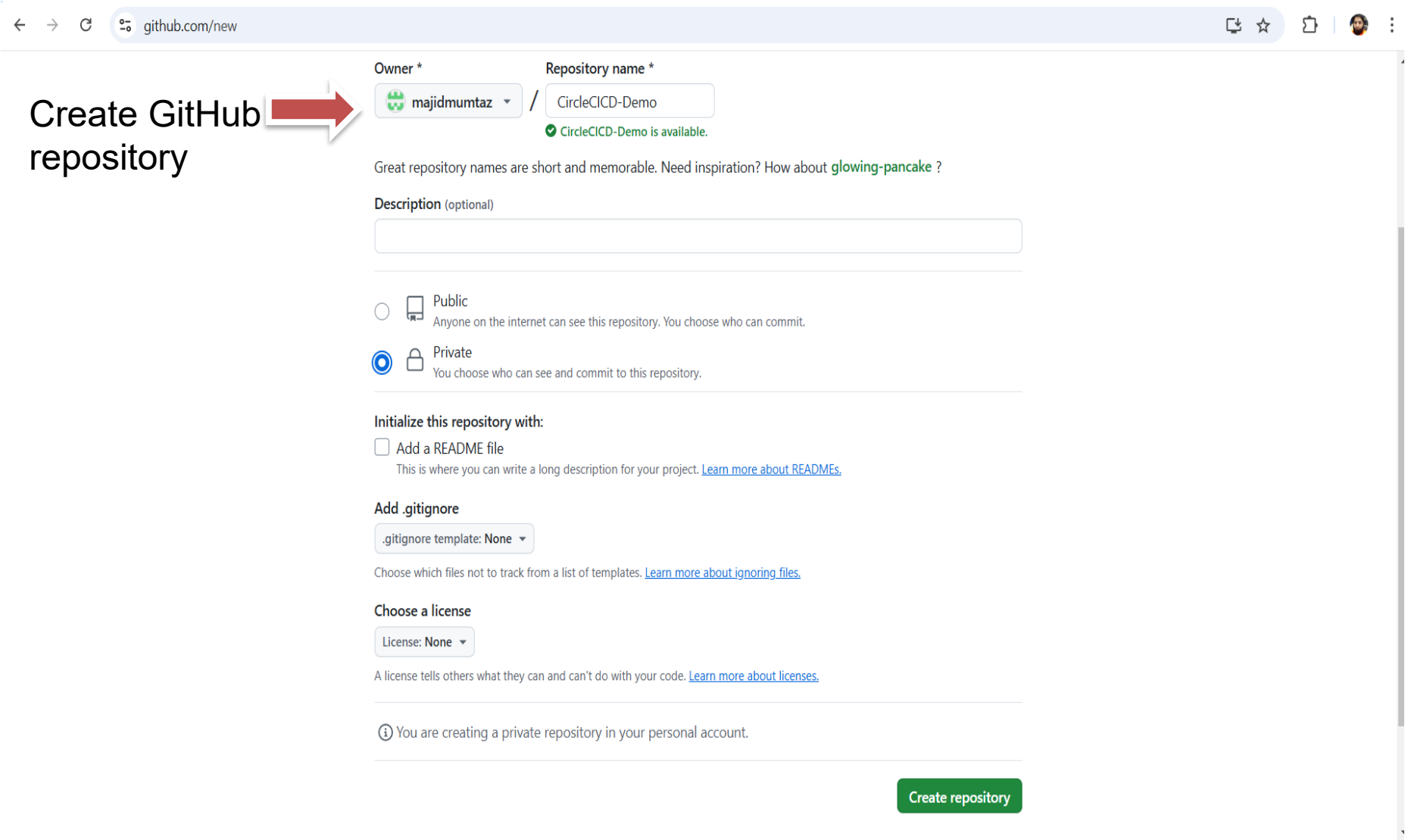
main-test.py > TestSubtract

```python
from main import Add, Subtract

def TestAdd():
        assert Add(2,3) == 5
        assert Add(5,5) == 10
        print("Add Function works correctly")

def TestSubtract():
        assert Subtract(5, 2) == 3
        print("Subtract Function works correctly")

if __name__ == '__main__':
        TestAdd()
        TestSubtract()
```

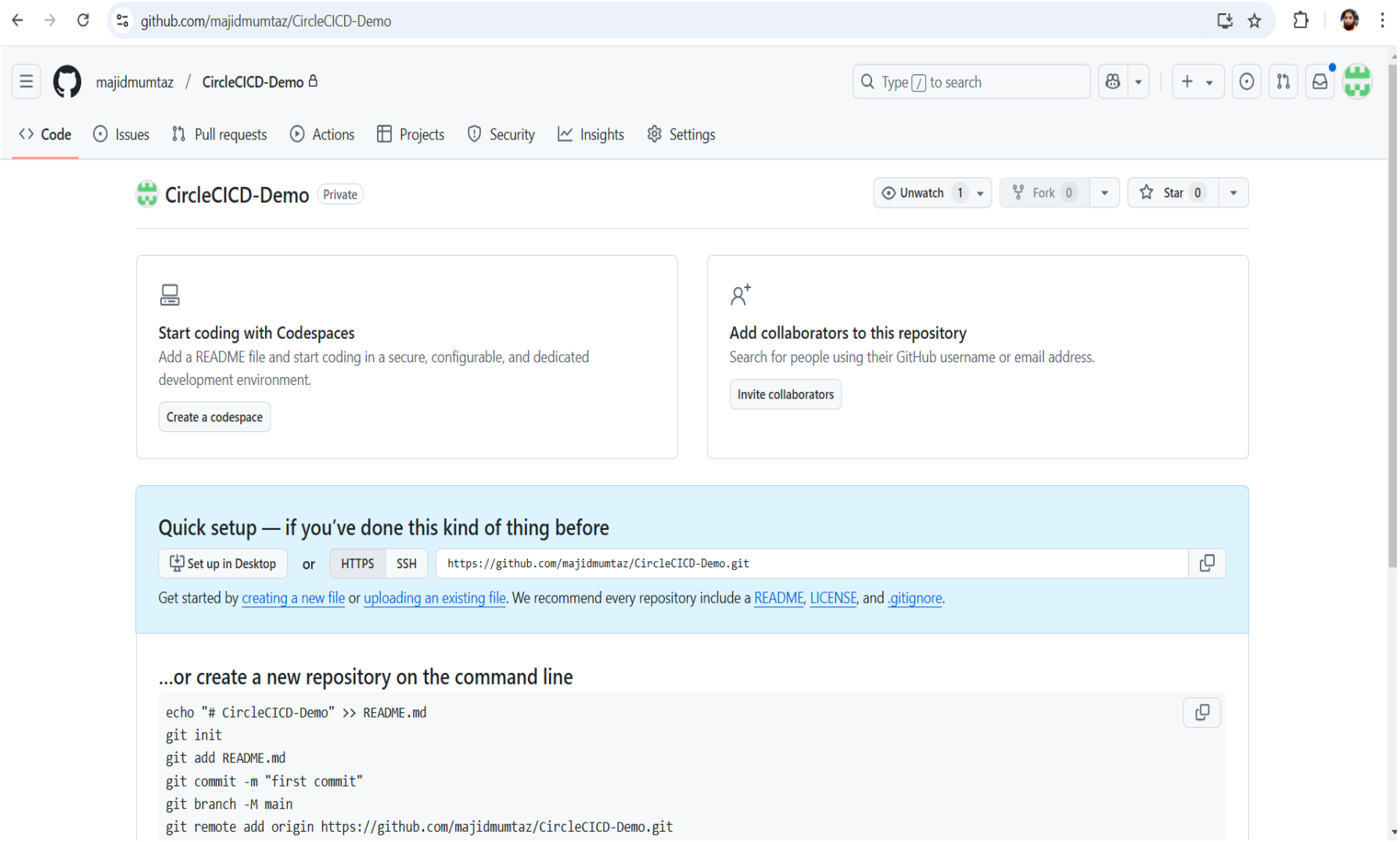
Example adapted from: https://www.srcmake.com/home/circleci

GitHub repository

Create GitHub repository



GitHub repository



GitHub repository

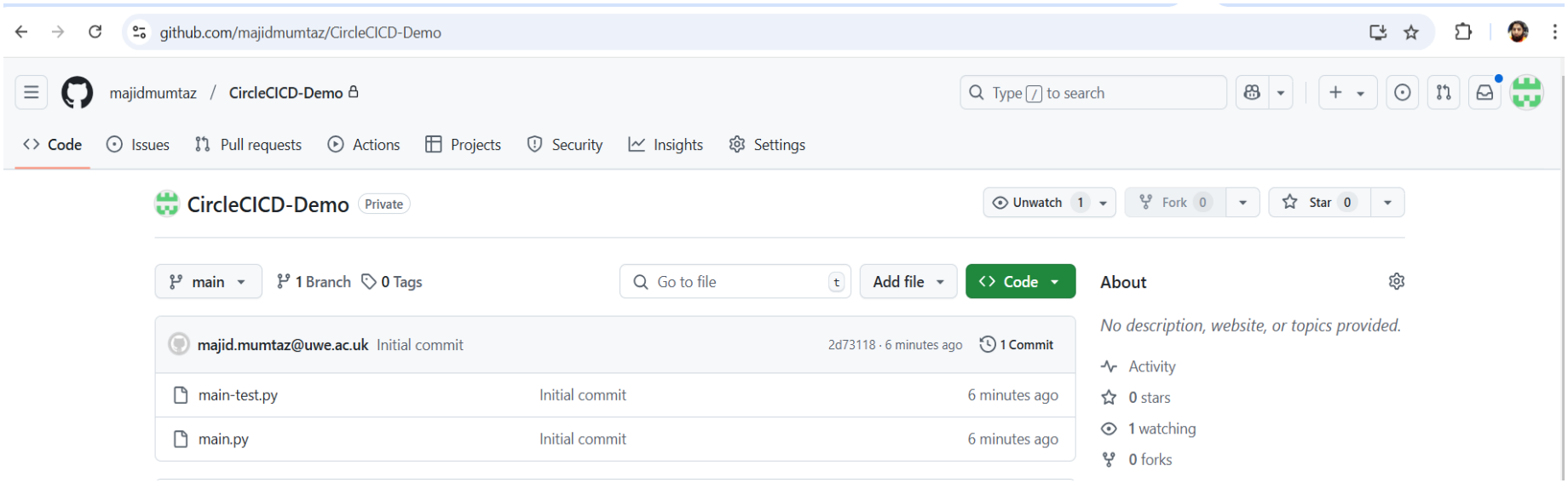
- Create local git repository, add, commit, to remotely add and push the code.

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO_NAME.git
git push -u origin main
```

```
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\CircleCICD-Demo\CircleCICD-Demo> git add .
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\CircleCICD-Demo\CircleCICD-Demo> git add .
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\CircleCICD-Demo\CircleCICD-Demo> git commit -m "Initial commit"
[main (root-commit) 2d73118] Initial commit
 2 files changed, 25 insertions(+)
 create mode 100644 main-test.py
 create mode 100644 main.py
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\CircleCICD-Demo\CircleCICD-Demo> git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 542 bytes | 180.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/majidmumtaz/CircleCICD-Demo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

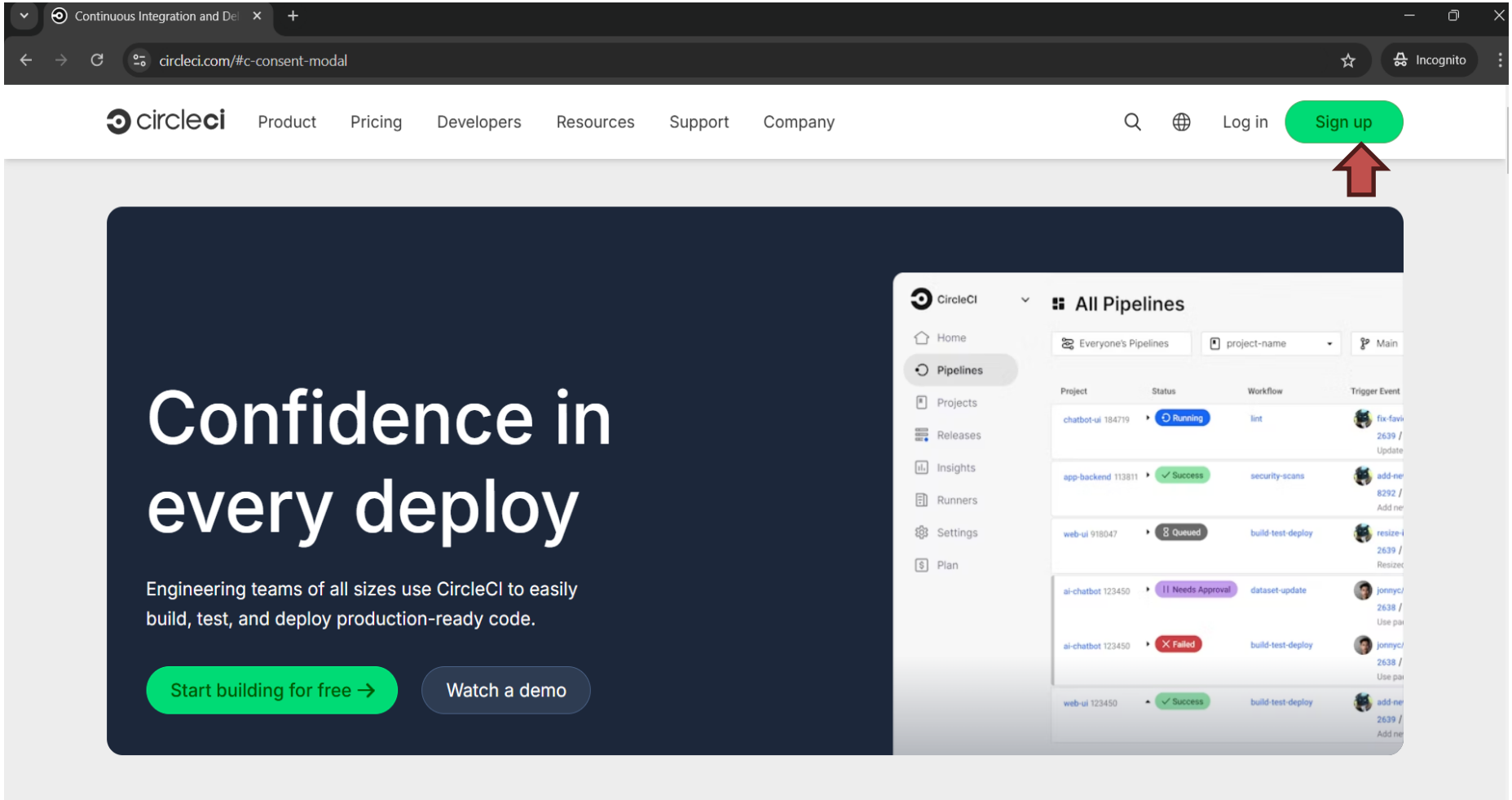
GitHub repository

This confirms the source code is successfully pushed in GitHub Repo.



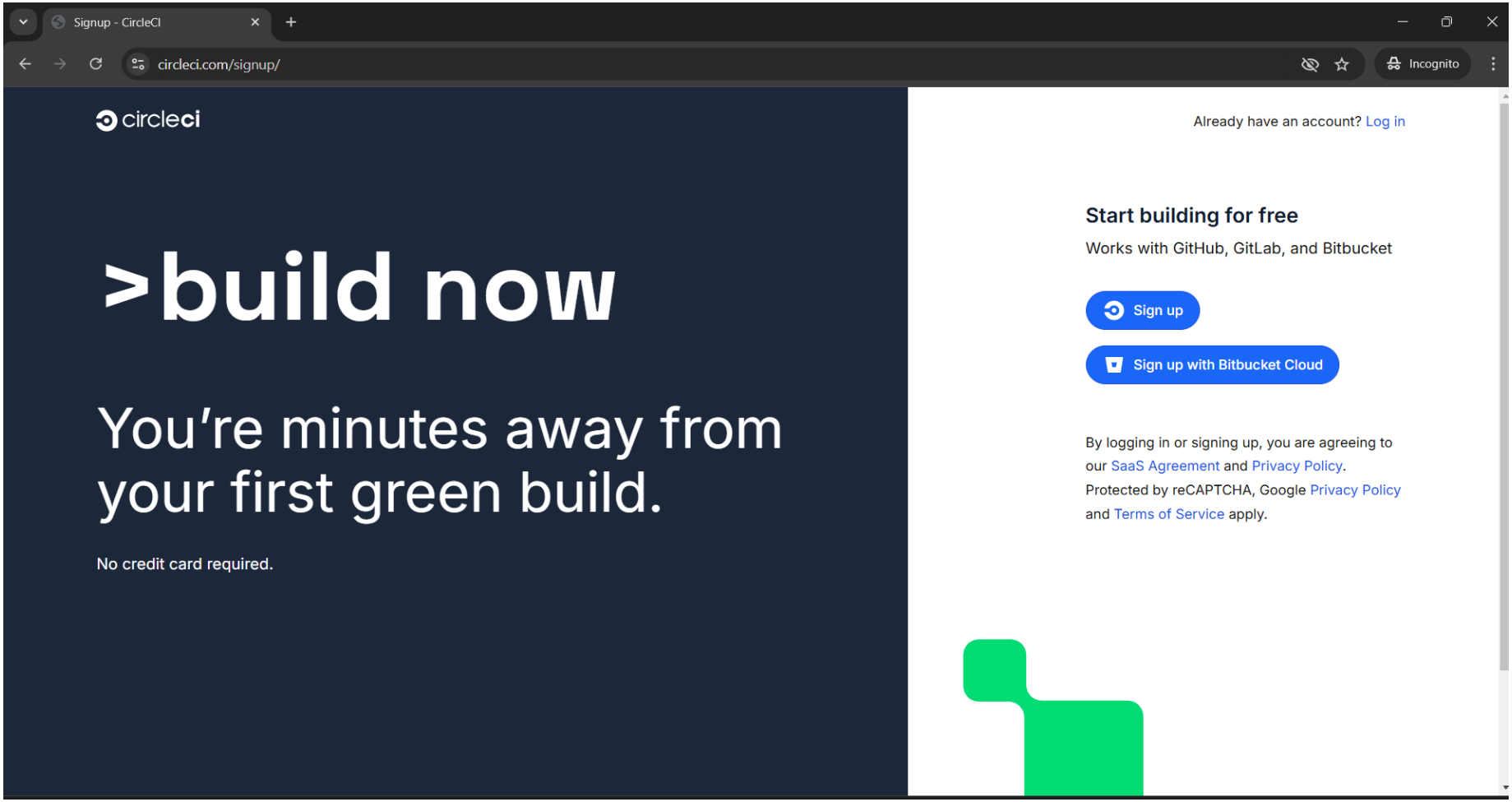
CircleCI

- Sign up at https://circleci.com if already not done so
- Follow the sign up process and connect with GitHub or GitLab
  - This may ask you to create SSH Keys (ssh-keygen) and copy/paste private key on CircleCI portal and public key on GitHub portal (use deploy keys guide)



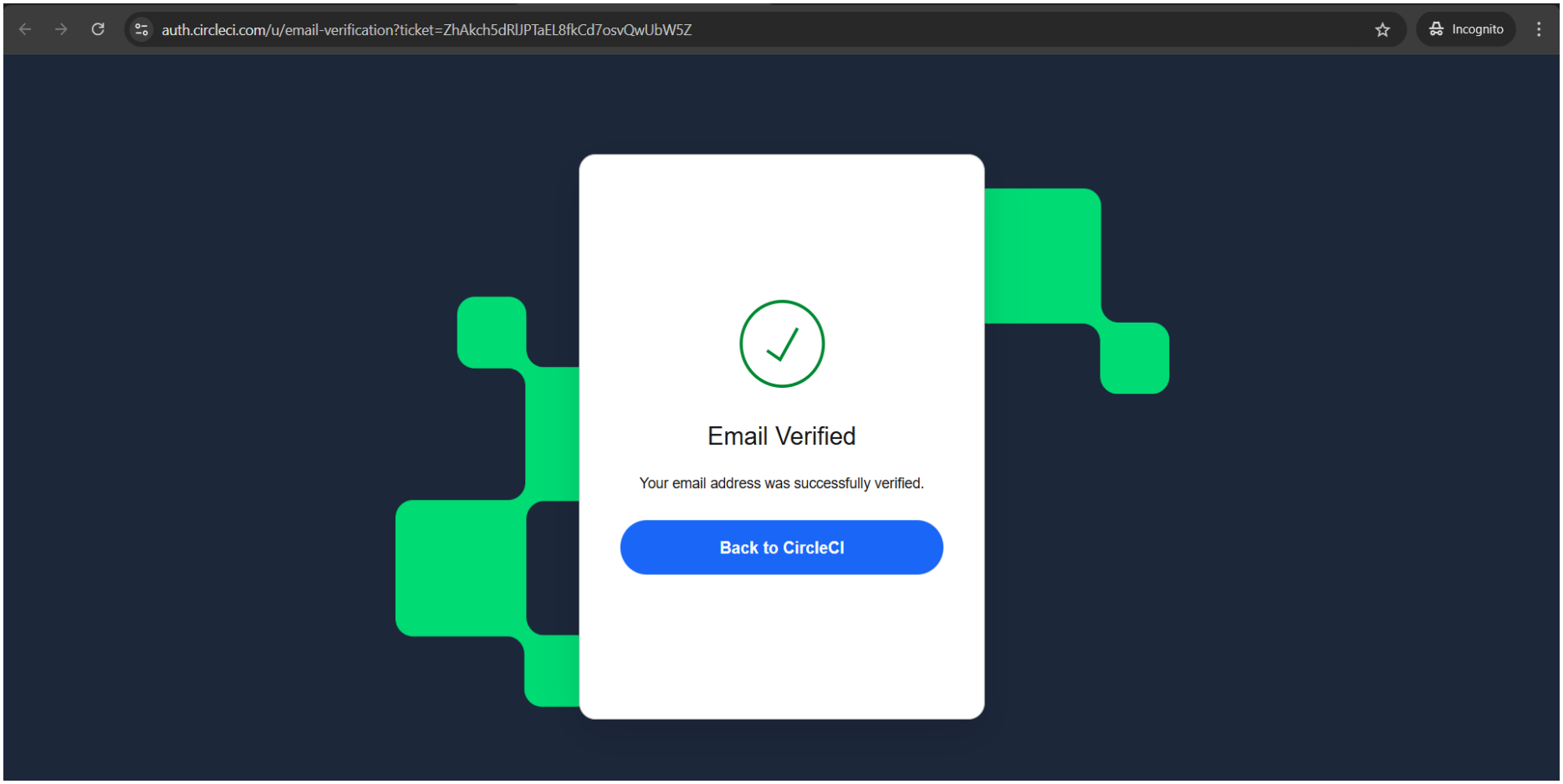
CircleCI

- Sign up at https://circleci.com if already not done so
- Follow the sign up process and connect with GitHub or GitLab
  - This may ask you to create SSH Keys (ssh-keygen) and copy/paste private key on CircleCI portal and public key on GitHub portal (use deploy keys guide)



CircleCI

- Sign up at https://circleci.com if already not done so
- Follow the sign up process and connect with GitHub or GitLab
  - This may ask you to create SSH Keys (ssh-keygen) and copy/paste private key on CircleCI portal and public key on GitHub portal (use deploy keys guide)



CircleCI

- Sign up at https://circleci.com if already not done so
- Follow the sign up process and connect with GitHub or GitLab
  - This may ask you to create SSH Keys (ssh-keygen) and copy/paste private key on CircleCI portal and public key on GitHub portal (use deploy keys guide)

```
C:\Users\m3-mumtaz>ssh-keygen -t ed25519 -C majid207@gmail.com
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\m3-mumtaz/.ssh/id_ed25519): circle_newkey
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in circle_newkey
Your public key has been saved in circle_newkey.pub
The key fingerprint is:
SHA256:3CjM2uCURQoKhf64VU6V9GN3EFj1QjztlSKcrbFjP20 majid207@gmail.com
The key's randomart image is:
+--[ED25519 256]--+
|.oo   o.. ++*o. .|
|o. . o o.. =o=.o.|
|o   . o  + .=o+..|
| .   B ..oo=. .. |
|  o B + S o o .  |
| . = = .     o E |
|  o o .       o  |
| .               |
|                 |
+----[SHA256]-----+

C:\Users\m3-mumtaz>type circle_newkey
```

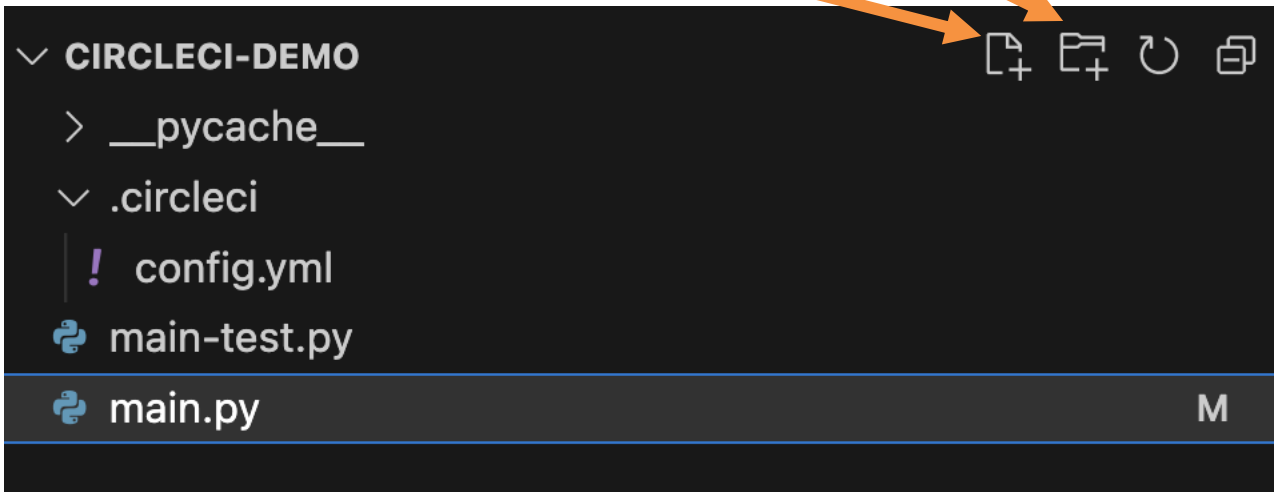
```
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW
Q[redacted]EKR0z
B[redacted]DiqlA
A[redacted]+3x9D
ftmdD8XpuTquMZu4OKqUAAAAEm1hamlkMjA3QGdtYWlsLmNvbQECAw==
-----END OPENSSH PRIVATE KEY-----

C:\Users\m3-mumtaz>type circle_newkey.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIE7KvlCkrrDLb2aolOB+3x9DftmdD8XpuTquMZu4OKqU majid207@gmail.com
```

Configuration file

- We need to create configuration file for CircleCI pipeline activation

Create directory in the name of .circleci

Create a file config.yml



Configuration file - Example

- YAML configuration guidelines - https://circleci.com/docs/introduction-to-yaml-configurations/

main.py | main-test.py | config.yml

.circleci > config.yml > ...

```yaml
version: 2.1  #circle ci version

jobs:   # There will be two jobs build and test
  build:
    working_directory: ~/circleci-python  #folder inside docker image
    docker:
      - image: "cimg/python:3.13.5"    #docker image with python 3.13.5
    steps:        #inside docker image folder then run these steps
      - checkout            #first step: checkout codebase from github to working directory
      - run: python3 main.py  #run python program
  test:
    working_directory: ~/circleci-python
    docker:
      - image: "cimg/python:3.13.5"
    steps:
      - checkout
      - run: python3 main-test.py

workflows:  #specify when to run these jobs and on which branches
  Open run panel to run this workflow
  build_and_test:
    jobs:
      - build
      - test:
          requires:
            - build  #only test if it is built
```

*Example adapted from: https://www.srcmake.com/home/circleci

Configuration file - Example

- YAML is quite picky about Indentations. You can use YAML Checker to parse your YAML and make sure it is valid.
- CircleCI fleet of images which you may use:
  - https://circleci.com/developer/images?imageType=docker
- CircleCI images documentation: https://circleci.com/docs/circleci-images/
- Using Docker Authenticated Pulls: https://circleci.com/docs/private-images/
  - Authenticated pulls allow access to private Docker images.

Configuration file

Welcome | main.py | main-test.py | config.yml

.circleci > config.yml > [ ] Workflows > build_and_test > [ ] Jobs > build

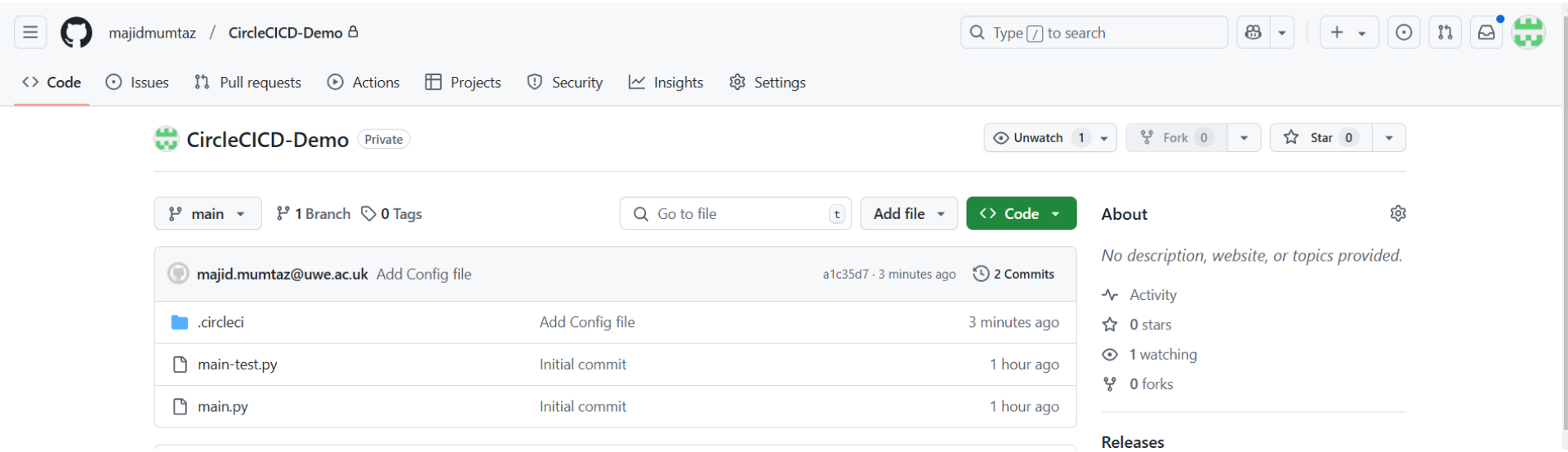
```yaml
version: 2.1  #circle ci version

jo

    docker:
      - image: "cimg/python:3.11.5"    #docker image with python 3.11.5
    steps:        #inside docker image folder then run these steps
      - checkout            #first step: checkout codebase from github to working directory
      - run: python3 main.py  #run python program
  test:
    working_directory: ~/circleci-python
    docker:
      - image: "cimg/python:3.11.5"
```

- Don’t forget to add & push configuration file

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | SQL CONSOLE | COMMENTS

```
To https://github.com/majidmumtaz/CircleCICD_Demo.git
 * [new branch]      main -> main
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\Teaching Block 2 Material\week7-new\CircleCICD_Demo\CircleCICD_Demo> git add .
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\Teaching Block 2 Material\week7-new\CircleCICD_Demo\CircleCICD_Demo> git commit -m "Add Circleci c
onfig file"
[main 732cf74] Add Circleci config file
 1 file changed, 25 insertions(+)
 create mode 100644 .circleci/config.yml
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\Teaching Block 2 Material\week7-new\CircleCICD_Demo\CircleCICD_Demo> git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 714 bytes | 64.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/majidmumtaz/CircleCICD_Demo.git
   916589a..732cf74  main -> main
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\Teaching Block 2 Material\week7-new\CircleCICD_Demo\CircleCICD_Demo>
```

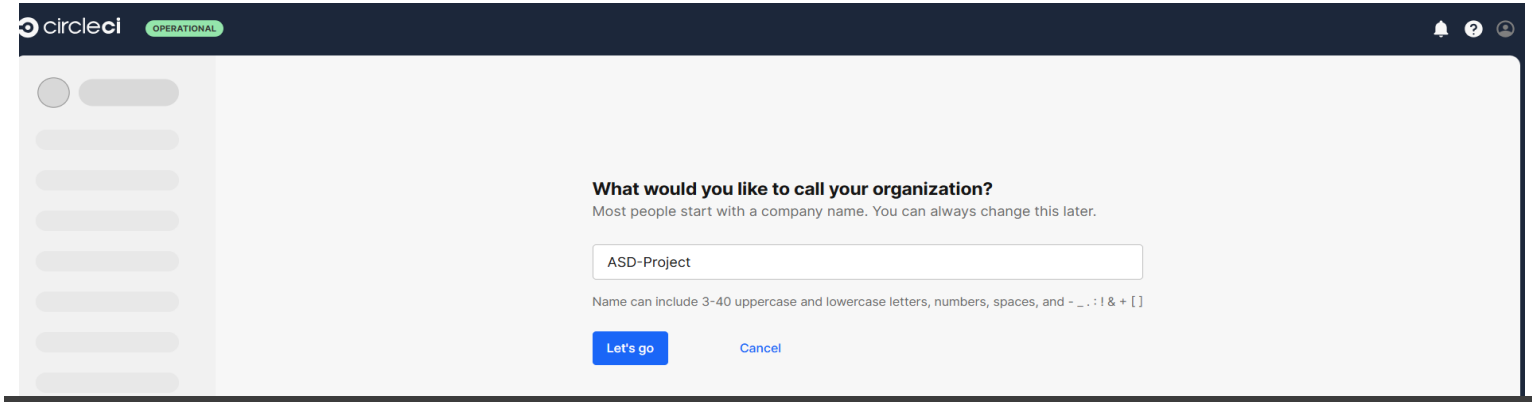
.CircleCI/config.yml pushed in repo.

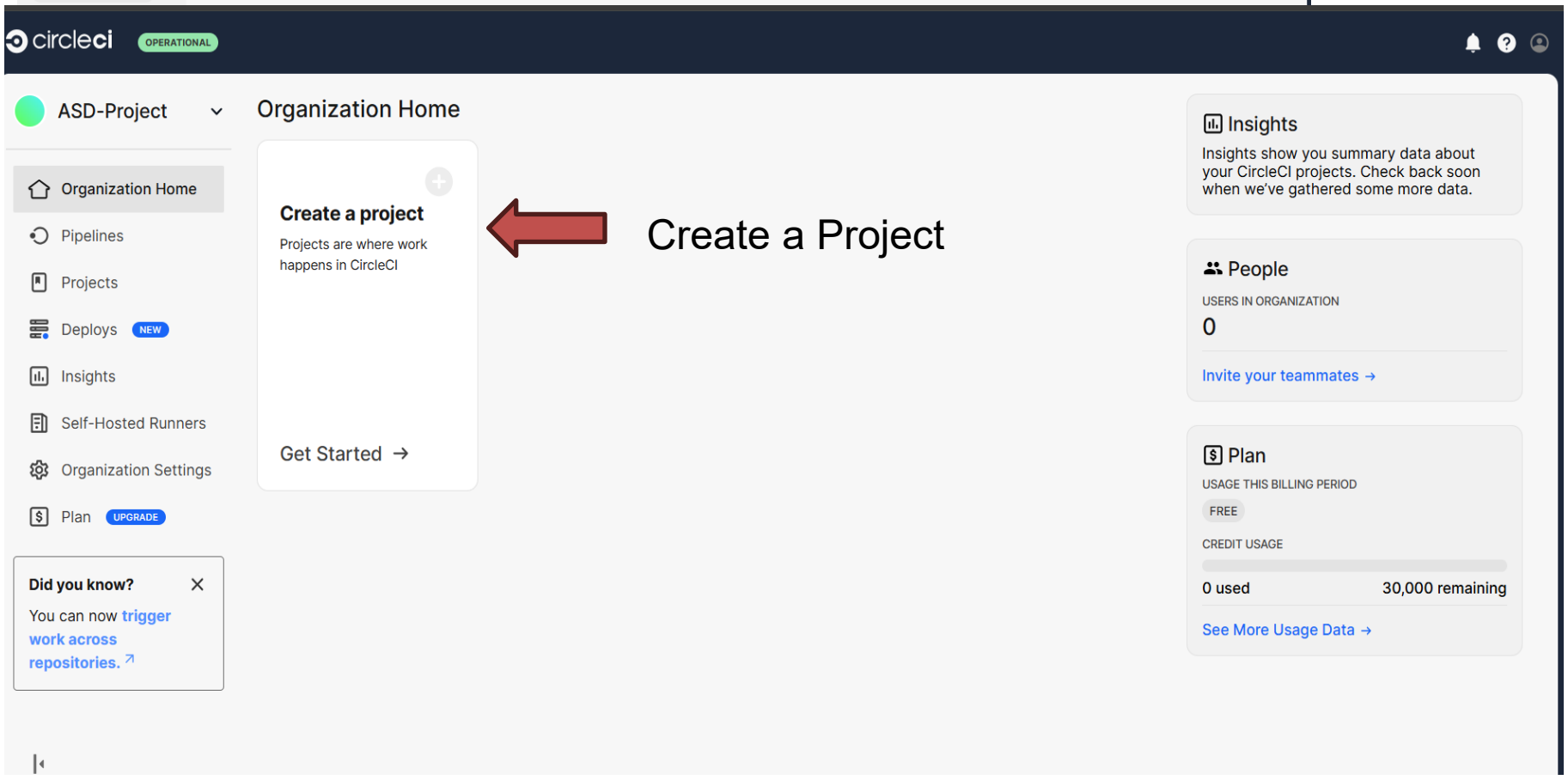


PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SQL CONSOLE COMMENTS powershell

```
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\CircleCICD-Demo\CircleCICD-Demo\.circleci> git add .
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\CircleCICD-Demo\CircleCICD-Demo\.circleci> git commit -m "Add Config file"
[main a1c35d7] Add Config file
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 .circleci/config.yml
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\CircleCICD-Demo\CircleCICD-Demo\.circleci> git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 939 bytes | 117.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/majidmumtaz/CircleCICD-Demo.git
   2d73118..a1c35d7  main -> main
```

CircleCI Web Frontend





Create a Project

CircleCI Project CI/CD Info

circleci OPERATIONAL

What would you like to do in your project?

Exit

Build, test, and deploy your software application

Build, test, **evaluate** and deploy your **GenAI-enabled** software application

Something else

circleci OPERATIONAL

1 Enter project details

2 Set up a pipeline

Choose a repo

Set up your config

Set up your triggers

3 Review and finish setup

Exit

Enter project details

We'll help you set up a project to build, test, and deploy a software application.

What would you like to name your project?

Project names must:
- be 3-40 characters long
- contain only letters, numbers, spaces, or - _ . : ! & + [ ]
- begin with a letter

← Go back

Next: set up a pipeline →

CircleCI Project Info & Pipeline

UWE Bristol University of the West of England

circleci OPERATIONAL

1 Enter project details
2 Set up a pipeline
Choose a repo
Set up your config
Set up your triggers
3 Review and finish setup

Enter project details

We'll help you set up a project to build, test, and deploy a software application.

What would you like to name your project?

CircleCICD-Demo

Project names must:
- be 3-40 characters long
- contain only letters, numbers, spaces, or - _ . : ! & + [ ]
- begin with a letter

← Go back Next: set up a pipeline →

circleci OPERATIONAL

Enter project details
2 Set up a pipeline
Choose a repo
Set up your config
Set up your triggers
3 Review and finish setup

Exit

Let's get a pipeline set up

First, a few details on how pipelines work. We'll help you set this all up.

CircleCI projects get work done by running **pipelines.**

Pipeline
Pipelines orchestrate executable commands and scripts for your CI/CD process

Code
Pipelines can checkout code from VCS repositories

Config
Config files use YAML to define what runs during your pipeline

Triggers
Triggers automatically run your pipelines when an event occurs

What would you like to name your project's first pipeline?

build-and-test

We suggest naming pipelines after the work they will perform, e.g. build-and-test

Next: choose a repo →

CircleCI Project repo Selection

circleci OPERATIONAL

- Enter project details
- 2 Set up a pipeline
  - Choose a repo
  - Set up your config
  - Set up your triggers
- 3 Review and finish setup

Exit

Let's choose a repo for your pipeline

The code from the repo that you choose here will be checked out when running the pipeline.

Which repo would you like to use in this pipeline?

Connect a VCS to choose a repo

This will give CircleCI access to repos that can be used in any project in your ASD-Project organization. You can add/remove the repos that CircleCI has access to later.

Connect to GitHub

Connect to GitLab Cloud

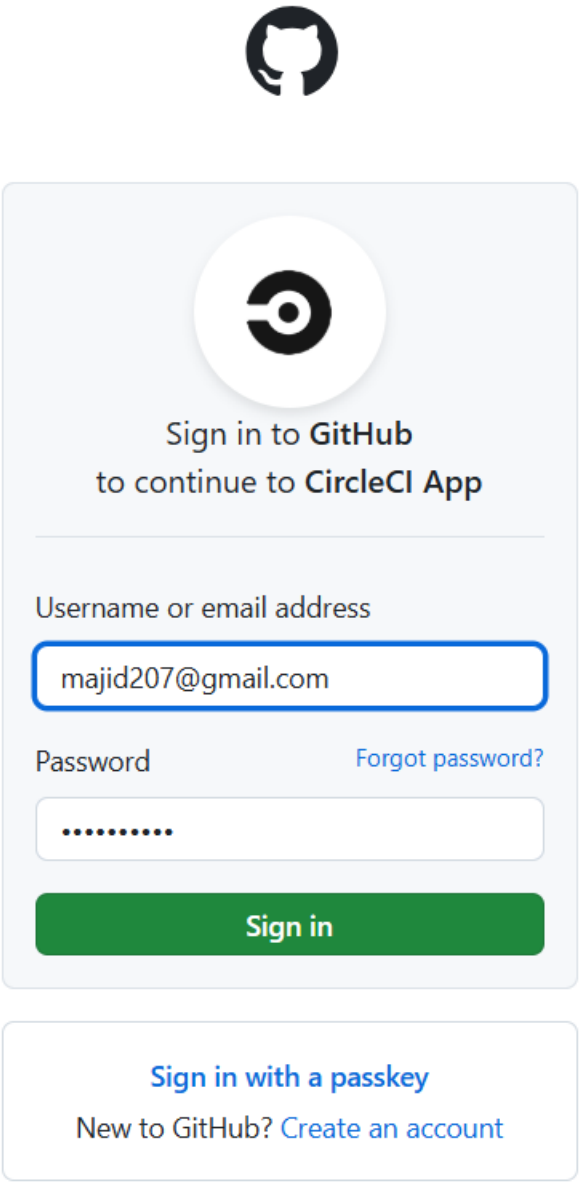
Connect to GitLab Self-managed

Connect to Bitbucket Data Center

← Go back

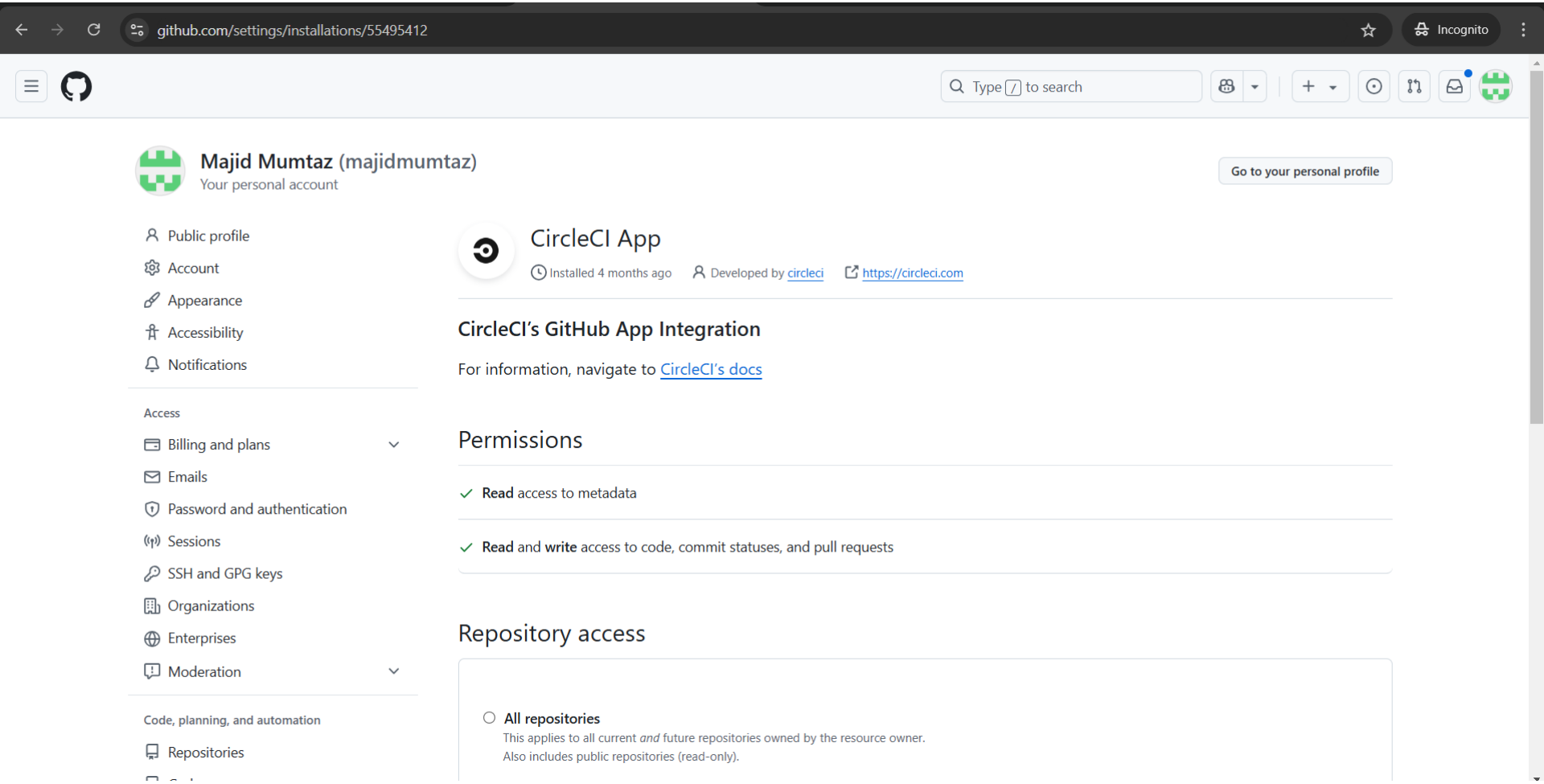
Next: set up your config →

CircleCI Project GitHub repo

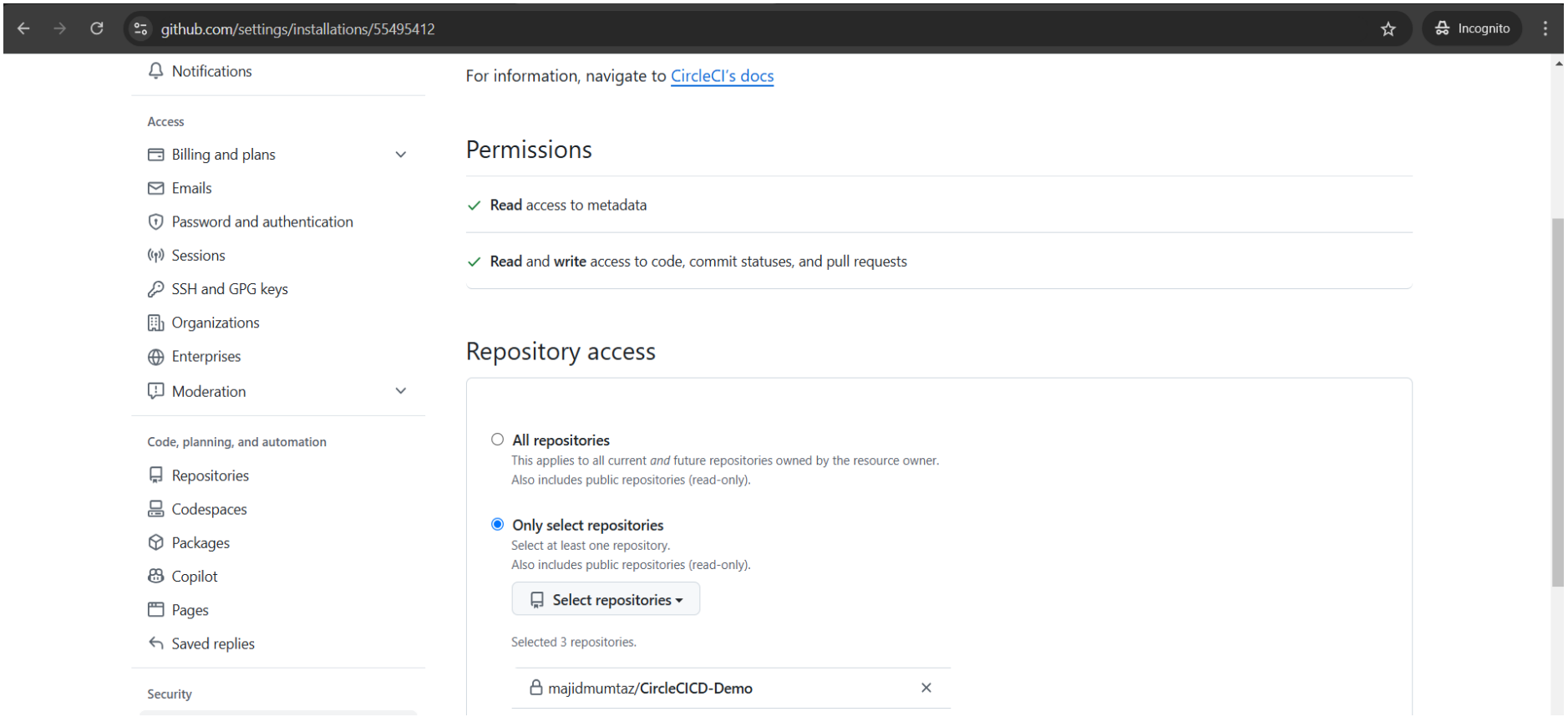


CircleCI Project repo permission

UWE Bristol University of the West of England



CircleCI Project repo permission



CircleCI Project Config file

circleci OPERATIONAL

- Enter project details
- 2 Set up a pipeline
  - Choose a repo
  - Set up your config
  - Set up your triggers
- 3 Review and finish setup

Exit

Let's define what runs in your pipeline

Looks like you already have a config file in CircleCICD_Demo called config.yml
We'll use this for your pipeline.

.circleci/config.yml

```yaml
version: 2.1  #circle ci version

jobs:   # There will be two jobs build and test
  build:
    working_directory: ~/circleci-python  #folder inside docker image
    docker:
      - image: "cimg/python:3.11.5"    #docker image with python 3.11.5
    steps:        #inside docker image folder then run these steps
      - checkout            #first step: checkout codebase from github to working directory
      - run: python3 main.py  #run python program
  test:
    working_directory: ~/circleci-python
    docker:
      - image: "cimg/python:3.11.5"
    steps:
      - checkout
```

← Go back Next: set up triggers →

CircleCI Project Trigger setup

circleci OPERATIONAL

- Enter project details
- 2 Set up a pipeline
  - Choose a repo
  - Set up your config
  - Set up your triggers
- 3 Review and finish setup

Exit

Let's set up your triggers

Run your pipeline anytime you like by adding triggers.

Triggers tell CircleCI when to run your pipeline.

Run **build-and-test**

When **Triggers**
Triggers automatically run your pipelines when an event occurs

We've created a GitHub trigger to get you started. If you don't need any additional triggers, click next.

When would you like to run your pipeline?

+ Add Trigger your pipeline from events sent by any 3rd party webhook

Github App Trigger
Commits pushed to any branch/tag in CircleCICD_Demo

← Go back | Next: review and finish setup →

CircleCI Project Finish setup

circleci OPERATIONAL

- Enter project details
- Set up a pipeline
  - Choose a repo
  - Set up your config
  - Set up your triggers
- 3 **Review and finish setup**

Exit

Review your setup

You're at the finish line! All that's left to do is review what you've set up.

Pipeline name
build-and-test

Code
CircleCICD_Demo

Config File
We'll use your existing config file

```
.circleci/config.yml
```

Triggers

Github App Trigger
Commits pushed to any branch/tag in CircleCICD_Demo

← Go back | Finish setup

CircleCI Workflow Success Email

Your workflow completed successfully.

Author	majidmumtaz
Project	ASD-UWE/CircleCICD_Demo
Workflow	build_and_test
Branch	main
Commit	Addition assertion check 3aa85da

Jobs	build	Success
	test	Success

Add new branch add-test to GitHub repo and pushed it

- Forked a branch **add-test**
- Changed the **main-test.py** code with an error assertion
- pushed it to the **add-test** branch on GitHub repository

```
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\Teaching Block 2 Material\week7-new\CircleCICD_Demo\CircleCICD_Demo> git checkout -b add-test
Switched to a new branch 'add-test'
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\Teaching Block 2 Material\week7-new\CircleCICD_Demo\CircleCICD_Demo> git add .
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\Teaching Block 2 Material\week7-new\CircleCICD_Demo\CircleCICD_Demo> git commit -m "Add test to ma
ke sure 6 + 5 is equal to 13"
[add-test d2464e4] Add test to make sure 6 + 5 is equal to 13
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\Teaching Block 2 Material\week7-new\CircleCICD_Demo\CircleCICD_Demo> git push --set-upstream origi
n add-test
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 354 bytes | 88.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'add-test' on GitHub by visiting:
remote:      https://github.com/majidmumtaz/CircleCICD_Demo/pull/new/add-test
remote:
To https://github.com/majidmumtaz/CircleCICD_Demo.git
 * [new branch]      add-test -> add-test
branch 'add-test' set up to track 'origin/add-test'.
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\Teaching Block 2 Material\week7-new\CircleCICD_Demo\CircleCICD_Demo>
```

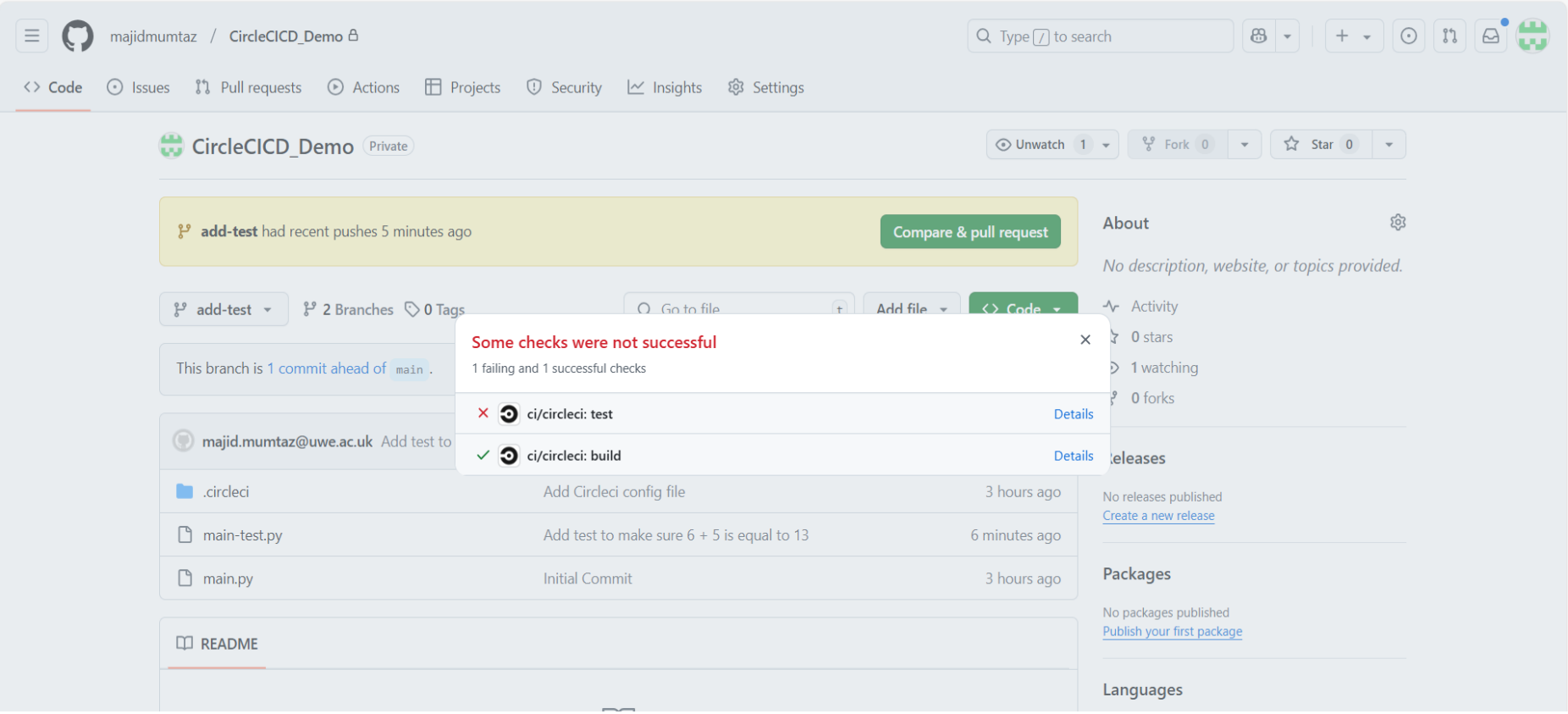
CircleCI workflow Failure Email

Uh-oh, this workflow did not succeed.

Author	majidmumtaz
Project	ASD-UWE/CircleCICD_Demo
Workflow	build_and_test
Branch	add-test
Commit	Add test to make sure 6 + 5 is equal to 13 d2464e4

Jobs	build	Success
	test	Failed

GitHub repo of Failure pipeline



CircleCI dashboard

circleci OPERATIONAL

ASD-UWE

Organization Home
Pipelines
Projects
Deploys NEW
Insights
Self-Hosted Runners
Organization Settings
Plan

More Flexible Requires
Requires now supports failed/canceled jobs.
Learn More

Dashboard > All Pipelines
Project > CircleCICD_Demo

CircleCICD_Demo

Trigger Pipeline | Project Settings

Filters: Everyone's Pipelines | CircleCICD_Demo | Select a Branch | All days

Auto-expand

Project	Status	Workflow	Trigger Event	Start	Duration	Actions
CircleCICD_Demo 2	Failed	build_and_test	add-test d2464e4 Add test to make sure 6 + 5 is equal to 13	11m ago	45s	
Jobs: build 3					24s	
Jobs: test 4					16s	
CircleCICD_Demo 1	Success	build_and_test	main 3aa85da Addition assertion check	3h ago	56s	
Jobs: build 1					47s	
Jobs: test 2					5s	

After correction add-test Branch push

After making Correction, Push again add-test branch.

```
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\Teaching Block 2 Material\week7-new\CircleCICD_Demo\CircleCICD_Demo> git add .
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\Teaching Block 2 Material\week7-new\CircleCICD_Demo\CircleCICD_Demo> git commit -m "Add correction
 and test Sum of 6 + 5 is equal to 11 not 13"
[add-test 0f94b56] Add correction and test Sum of 6 + 5 is equal to 11 not 13
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\Teaching Block 2 Material\week7-new\CircleCICD_Demo\CircleCICD_Demo> git push --set-upstream origi
n add-test
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 368 bytes | 184.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/majidmumtaz/CircleCICD_Demo.git
   d2464e4..0f94b56  add-test -> add-test
branch 'add-test' set up to track 'origin/add-test'.
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_Modules\ASD TB2\Teaching Block 2 Material\week7-new\CircleCICD_Demo\CircleCICD_Demo>
```

CircleCI dashboard

circleci OPERATIONAL

ASD-UWE

Organization Home
Pipelines
Projects
Deploys NEW
Insights
Self-Hosted Runners
Organization Settings
Plan

More Flexible Requires
Requires now supports failed/canceled jobs.
Learn More

CircleCICD_Demo

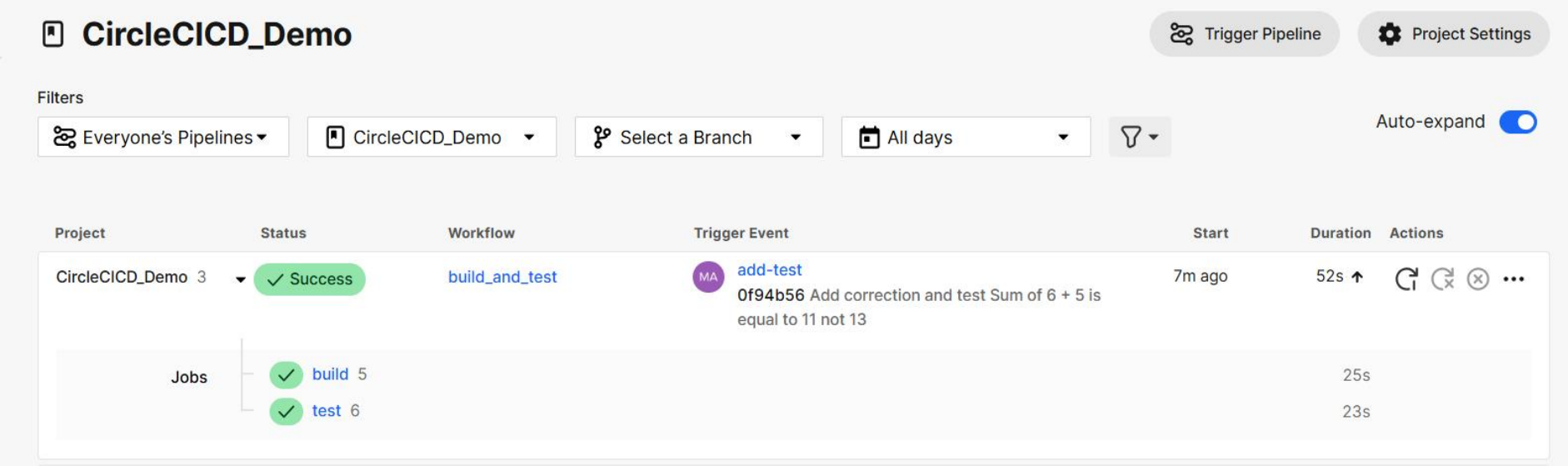
Trigger Pipeline | Project Settings

Filters: Everyone's Pipelines | CircleCICD_Demo | Select a Branch | All days

Auto-expand

Project	Status	Workflow	Trigger Event	Start	Duration	Actions
CircleCICD_Demo 3	Success	build_and_test	add-test 0f94b56 Add correction and test Sum of 6 + 5 is equal to 11 not 13	4m ago	52s	
Jobs	build 5				25s	
	test 6				23s	
CircleCICD_Demo 2	Failed	build_and_test	add-test d2464e4 Add test to make sure 6 + 5 is equal to 13	20m ago	45s	
Jobs	build 3				24s	
	test 4				16s	
CircleCICD_Demo 1	Success	build_and_test	main 3aa85da Addition assertion check	3h ago	56s	
Jobs	build 1				47s	
	test 2				5s	

CircleCI demo project jobs



CircleCI demo Project Build job

Dashboard > Project > Branch > Workflow > Job

All Pipelines > CircleCICD_Demo > add-test > build_and_test > build (5)

build Success

Rerun

Duration / Finished	Queued	Executor / Resource Class	Branch	Commit	Author & Message
24s / 8m ago	0s	Docker / Large	add-test	0f94b56	Add correction and test Sum of 6 + 5 is equal to 11 not 13

STEPS TESTS TIMING ARTIFACTS RESOURCES

Parallel runs

0 00:24

Use parallelism to run faster tests
Parallelism speeds up tests by splitting them across multiple executors.

Go to Docs

Spin up environment	23s
Preparing environment variables	0s
Checkout code	0s
python3 main.py	0s

CircleCI demo Project Test job

Dashboard > Project > Branch > Workflow > Job

All Pipelines > CircleCICD_Demo > add-test > build_and_test > test (6)

test Success

Rerun

Duration / Finished	Queued	Executor / Resource Class	Branch	Commit	Author & Message
22s / 10m ago	0s	Docker / Large	add-test	0f94b56	Add correction and test Sum of 6 + 5 is equal to 11 not 13

STEPS TESTS TIMING ARTIFACTS RESOURCES

Parallel runs

0 00:22

Use parallelism to run faster tests
Parallelism speeds up tests by splitting them across multiple executors.

Go to Docs

Spin up environment	21s
Preparing environment variables	0s
Checkout code	0s
python3 main-test.py	0s

Sources

- CircleCI Docs: https://circleci.com/docs/hello-world/
- CI/CD Tutorial / Demo - https://www.srcmake.com/home/circleci

GitHub Actions for CI

- **Create GitHub Actions Workflow Directory**
  - mkdir -p .github/workflows
- **Create the CI/CD Workflow File**
  - Create a file named .github/workflows/ci-cd.yml with the following content:

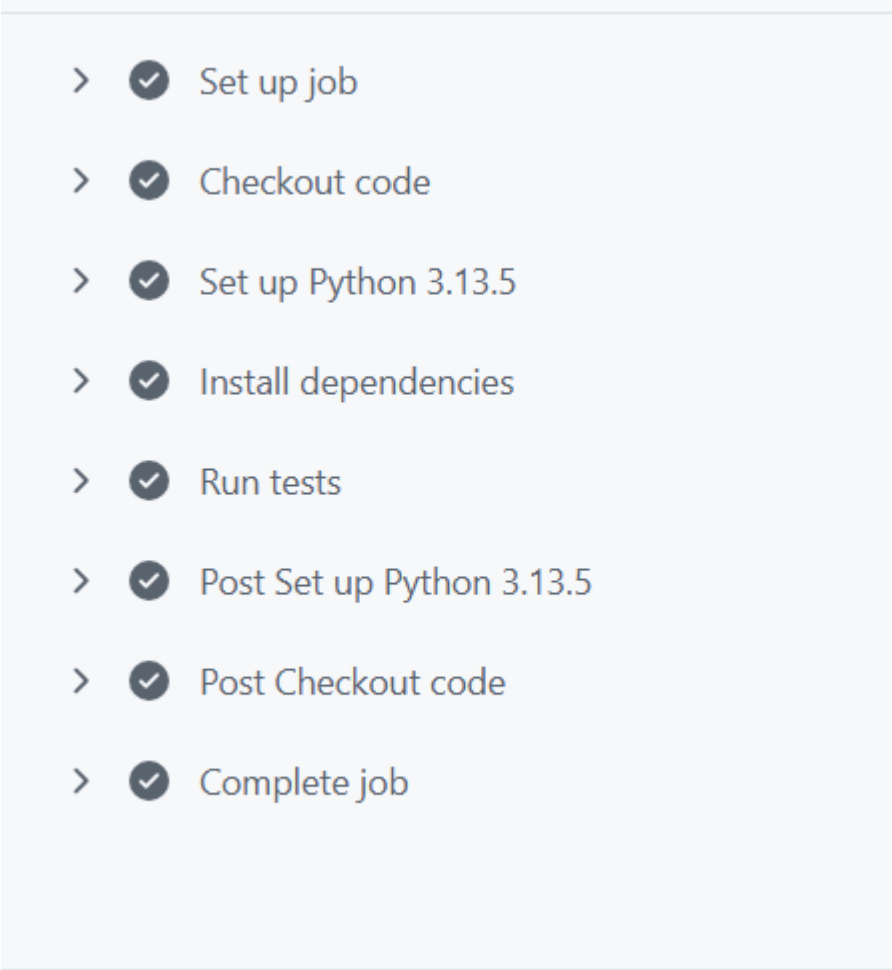
.github > workflows > ci-cd.yml

```yaml
name: Python CI/CD

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up Python 3.13.5
      uses: actions/setup-python@v5
      with:
        python-version: '3.13.5'

    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
        pip install -r requirements.txt

    - name: Run tests
      run: python main-test.py
```

GitHub Actions for CI (Conti.)

- **Push the Workflow Files**
  - git add .github/workflows/
  - git commit -m "Add GitHub Actions CI/CD pipeline"
  - git push origin main

- **Monitor the Pipeline Execution**
  - Go to your GitHub repository
  - Click on the "Actions" tab
  - You'll see your workflow running automatically
  - Click on the running workflow to see real-time logs



UWE Bristol University of the West of England

We looked at CI i.e. Build and Test.

What about CD?

CD

- Deployment: https://circleci.com/docs/deployment-overview/
- After developing and testing, software application needs to be deployed to make it available to its users
- Deployment targets may vary from one project to another
- So, you need to customize the configuration file to ensure a complete fully automated pipeline can be applied.

CD

- Deployment configuration requires
  - Deployment job in **config.yml**
  - Deployment parameters e.g., environment variables, SSH keys

- CircleCI offers:
  - Orbs: https://circleci.com/developer/orbs are packages with reusable configuration and can be used for deployment. Examples:
    - circleci/node@5.1.0, circleci/aws-cli@4.1.1, circleci/python@2.1.1
  - Convenience images: https://circleci.com/developer/images?imageType=docker for use with Docker that contain tools for common deployment scenarios. Examples:
    - cimg/deploy, cimg/azure, cimg/gcp, cimg/aws

CD

Dashboard | Project | Branch | Workflow

All Pipelines > circleci-demo > main > build_and_test_and_deploy

build_and_test_and_deploy Failed

Duration / Finished	Branch	Commit	Author & Message
14s / 21s ago	main	cdd9b30	Testing heroku deployment by changing config.yml

build 3s — test 3s — deploy 2s

CD Configuration (yml) file

main.py | main-test.py | config.yml

.circleci > config.yml

```yaml
version: 2.1  #circle ci version

jobs:   # There will be two jobs build and test
  build:
    working_directory: ~/circleci-python  #folder inside docker image
    docker:
      - image: "cimg/python:3.11.5"    #docker image with python 3.11.5
    steps:        #inside docker image folder then run these steps
      - checkout            #first step: checkout codebase from github to working directory
      - run: python3 main.py  #run python program
  test:
    working_directory: ~/circleci-python
    docker:
      - image: "cimg/python:3.11.5"
    steps:
      - checkout
      - run: python3 main-test.py

  deploy:
    docker:
      - image: cimg/deploy:2023.09
    steps:
      - checkout
      - run: echo "Hello World"
      - run: python3 main.py

workflows:  #specify when to run these jobs and on which branches
  build_and_test_and_deploy:
    jobs:
      - build
      - test:
          requires:
            - build  #only test if it is built
      - deploy:
          requires:
            - test #only deploy if the test has completed
          filters:
            branches:
              only: main # only deploy when on main
```

CD

Dashboard Project

All Pipelines > CircleCICD_Demo

CircleCICD_Demo

Trigger Pipeline | Project Settings

Filters

Everyone's Pipelines | CircleCICD_Demo | Select a Branch | All days

Auto-expand

Project	Status	Workflow	Trigger Event	Start	Duration	Actions
CircleCICD_Demo 5	Running	build_and_test_and_deploy	MA main 3f52c20 Merge pull request #1 from majidmumtaz/add-test Add test	1m ago	41s	
Jobs	build 9				17s	
	test 10				4s	
	deploy 11				14s	

CD

circleci OPERATIONAL

ASD-UWE

Organization Home
Pipelines
Projects
Deploys NEW
Insights
Self-Hosted Runners
Organization Settings
Plan

More Flexible Requires
Requires now supports failed/canceled jobs.
Learn More

Dashboard: All Pipelines > Project: CircleCICD_Demo > Branch: main > Workflow: build_and_test_and_deploy

build_and_test_and_deploy Success

Insights | Rerun

Duration / Finished	Branch	Commit	Author & Message
48s / 1m ago	main	3f52c20	Merge pull request #1 from majidmumtaz/add-test Add test

build 17s — test 4s — deploy 21s

CD Success message

Your workflow completed successfully.

Author	majidmumtaz
Project	ASD-UWE/CircleCICD_Demo
Workflow	build_and_test_and_deploy
Branch	main
Commit	Merge pull request #1 from majidmumtaz/add-test Add test 3f52c20

Jobs	build	Success
	test	Success
	deploy	Success

Summary

- You'll encounter Git and CI/CD a lot in software development
  - Continuous Integration
  - Continuous Deployment
  - GitHub and CircleCI

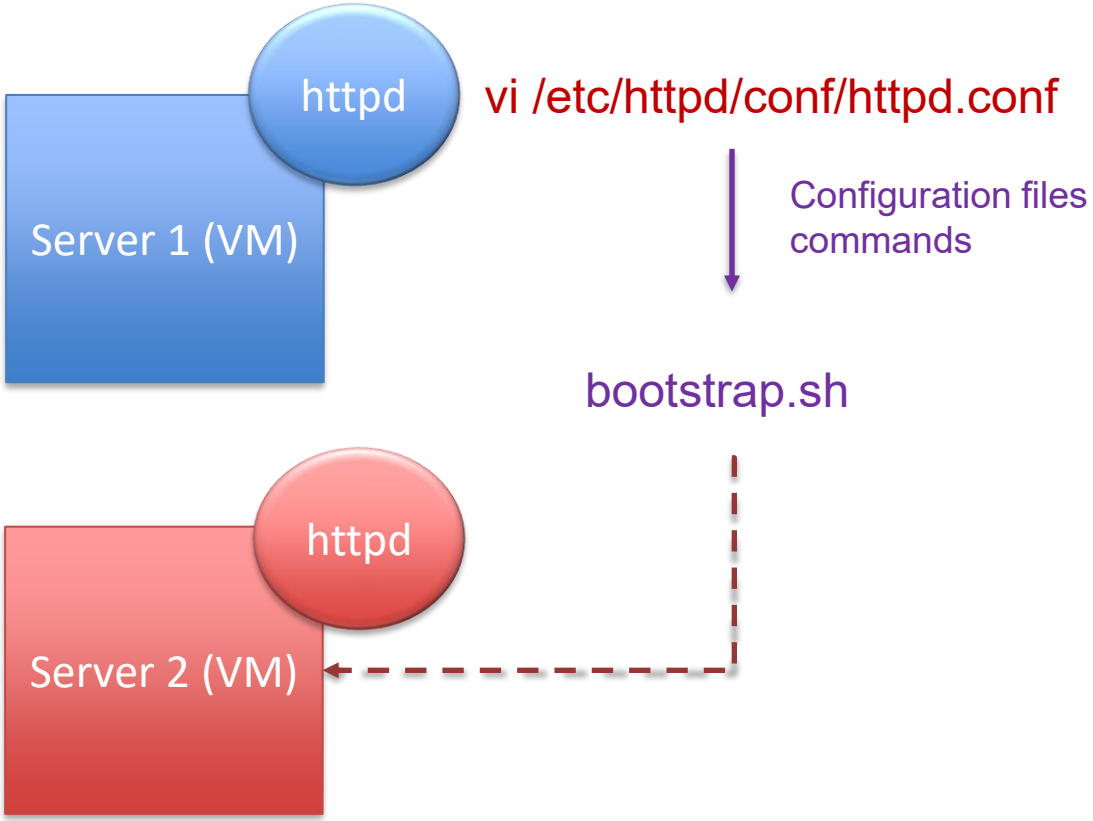
- There is a lot to learn

Check MLOPs

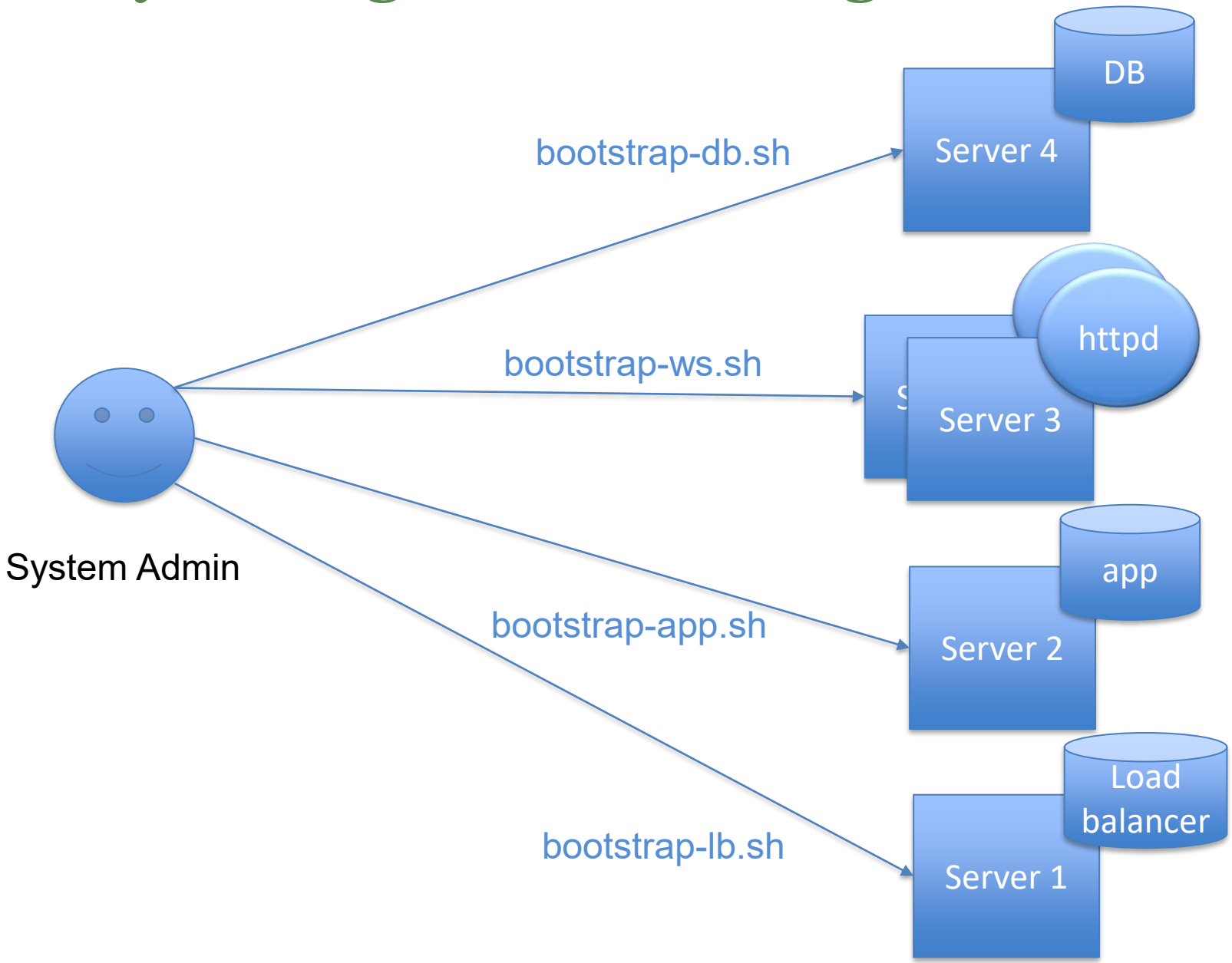
Check DevSecOPs

Configuration management

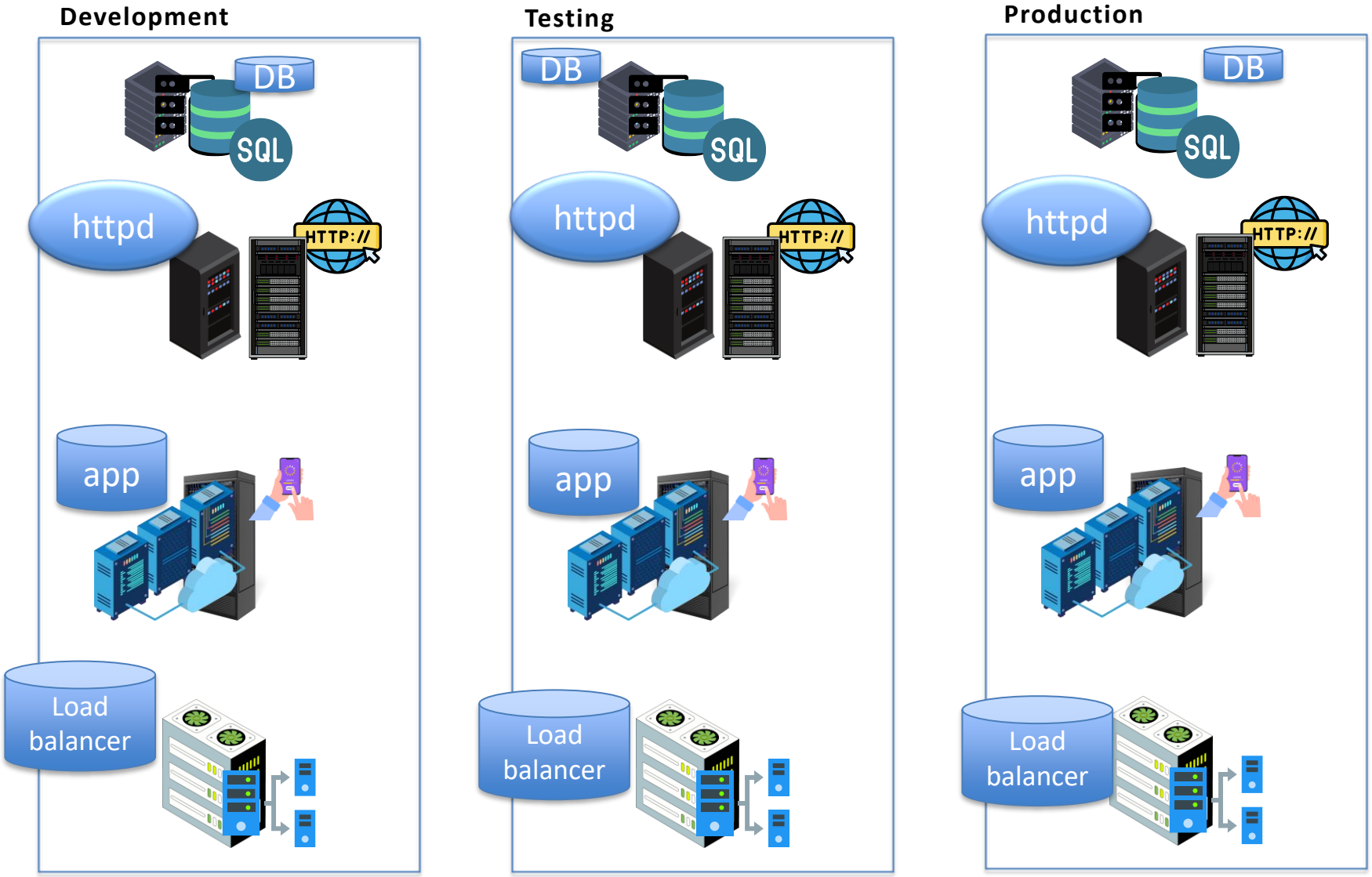
Why Configuration management?



Why Configuration management?



Why Configuration management?



Configuration management

- Configuration management addresses scalability of significant and repetitive tasks such as installations, upgrades, patches (e.g., security), user management etc
- State of system that can be distributed and deployed on other machines using scripts, if needed
  - E.g., deploying same server configuration several times

- DevOps teams can save time by automating such tasks in an IT environment

- A key enabler for CI/CD pipeline is infrastructure configuration a.k.a infrastructure as a code (IaC)

Configuration management

- Infrastructure-as-Code (IaC) is about managing and provisioning of infrastructure through configuration code instead of manual processes

'Source: Red hat'

'Source: https://opensource.com/article/17/11/continuous-infrastructure-other-ci '

  - Creates configuration files that contains state/specification of infrastructure (e.g., a Database server)
  - version control of configuration files should just be like other software source code file

- Self Read: https://geekflare.com/config-management-tools/

How can we do configuration management?

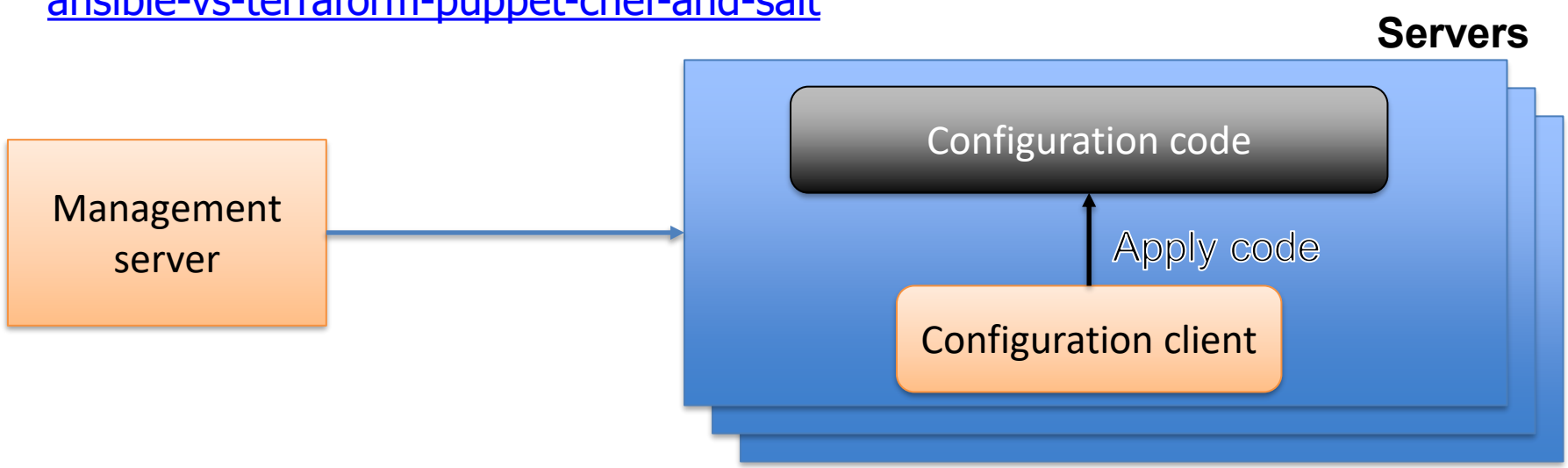
- **Two approaches:**
  - **Declarative approach:** defines the desire **state of the system** and required resources and associated expected properties

```
file { '/opt/app':
    ensure => 'directory',
    group => 'softwaredevelopers',
    mode => '0644',
}
```

  - **Imperative approach:** defines specific commands (in an order) needed to achieve the desired configuration
  - *#!/bin/sh*
  - *mkdir –p /opt/app*
  - *chown :softwaredevelopers /opt/app*
  - *chmod 777 /opt/app*

How can we do configuration management?

- There are Configuration management Tools
  - **Ansible** is suite of software tools, is open source and it makes IaC convenient
  - **Chef** is an IaC tool that automates configuration management
  - **Puppet** (declarative) is open source software configuration management tool
  - **CFEngine** is open source and automates configuration management
  - **Terraform** is open-source and makes IaC more convenient and easier. It is open source tool to create and change infrastructure through scripts.
- **Self Read:** https://www.redhat.com/en/topics/automation/understanding-ansible-vs-terraform-puppet-chef-and-salt



Summary

- Automation in software development can save a lot of time and resources
- We covered CI/CD pipeline using CircleCI and there are many other tools available
- We briefly covered configuration management
- Both are related to DevOps
- A lot of scope to learn for software developers to efficiently develop software without worrying about or spending unnecessary time on manual tasks
- As a software developer you should get yourself familiar with CI/CD and configuration management tools

- Sources:
- Configuration management - https://geekflare.com/config-management-tools/
- CM quick intro: https://www.youtube.com/watch?v=XZnwIvuKgvM&ab_channel=mkdev
- Tools: https://www.redhat.com/en/topics/automation/understanding-ansible-vs-terraform-puppet-chef-and-salt
- Continuous Infrastructure - https://opensource.com/article/17/11/continuous-infrastructure-other-ci